CME 213 Final Project Milestone 2
Real-Time Multi-Camera 3D Perception and Tracking for Robotics

Frances Raphael
fraphael@stanford.edu

Aanya Tashfeen
aanya@stanford.edu

1 Project Scope and Problem Definition

For this project, we are building a real-time multi-camera perception system for robotics. The goal is to take image streams from multiple cameras, estimate useful 3D information, and track objects over time. The input to the system will be recorded or simulated image sequences from two or more cameras. The output will be either a depth/disparity estimate, a set of tracked 2D/3D object locations, or both, depending on how much of the full pipeline we are able to complete.

The main computational problem is that each camera produces a large stream of image data, and many of the operations needed for perception are expensive. These include preprocessing, filtering, feature extraction, stereo or multi-view matching, and tracking. We will focus on implementing a simplified but complete version of the pipeline rather than trying to build a full robotics perception stack. A successful project for us would be an end-to-end system with a CPU baseline, a CUDA-accelerated version of the main vision kernel, and an MPI implementation that distributes work across camera streams or image partitions. We also want to clearly measure where the bottlenecks are and whether the system is limited more by computation, memory bandwidth, or communication overhead.

2 Survey of Related Work

Computer vision is a natural fit for GPU acceleration because many image operations apply similar computations across many pixels. Cervera discusses how OpenCV and CUDA can be used together to improve throughput in robotics vision systems, especially because image processing contains many massively parallel operations (Cervera, 2020). Huang et al. also show that vision algorithms can be mapped effectively to CUDA, reporting large speedups for motion tracking workloads compared with CPU implementations (Huang et al., 2007). These works support our plan to use CUDA for the dense image-processing parts of our pipeline.

Other work has explored GPU acceleration for object detection and classification. Harvey implements CUDA versions of several object classification algorithms and shows that GPU acceleration can produce major speedups while preserving comparable accuracy (Harvey, 2009). Afif et al. also present CUDA implementations of common computer vision algorithms and emphasize that feature extraction, filtering, and other image operations are good GPU targets because of their high parallelism (Afif et al., 2020). These papers are useful for our project because they motivate focusing on a small number of expensive kernels rather than trying to accelerate the entire application at once.

For depth estimation, multi-view stereo methods provide the foundation for recovering 3D structure from multiple images. MVSNet builds a cost volume from multiple views and performs depth inference using learned features and 3D convolution (Yao et al., 2018). More recent work such as MaGNet improves multi-view depth estimation by combining single-view depth probability with multi-view geometry, reducing the number of depth candidates that need to be evaluated (Bae et al., 2022). We are not planning to reproduce these deep-learning systems, but they are helpful

references because they show the main computational challenge in multi-view depth: matching information across views while managing memory and runtime.

Compared with these prior systems, our focus is not to advance the state of the art in depth estimation or tracking. Instead, our goal is to implement a simplified version of the same kind of workload and study how CUDA and MPI affect performance. Our implementation will likely use simpler stereo matching, filtering, or feature-based matching, but we will use the structure of these papers to justify our pipeline and performance choices.

3 Computational Challenges

The first major challenge is memory bandwidth. Image-based algorithms often require reading neighboring pixels or comparing patches across images, which can lead to many global memory accesses. If we implement stereo matching, each pixel may need to compare a window across multiple disparity candidates, so the amount of memory traffic can grow quickly.

The second challenge is choosing the right level of simplification. A full multi-camera detection, depth, tracking, and fusion system could become too large for the project timeline. To manage this, we will start with a minimal pipeline and then add complexity only if the baseline works.

The third challenge is MPI communication overhead. If each MPI process handles one camera or one image partition, the processes eventually need to exchange intermediate results. For example, depth estimation may require data from multiple views, and fusion requires combining results from different processes. We will need to measure whether this communication is small compared with GPU computation or whether it becomes the bottleneck.

4 CUDA Implementation Plan

We plan to implement the most computationally expensive vision step as one or more CUDA kernels. Our first target will be either image filtering/preprocessing or a simplified stereo matching kernel. For stereo matching, each thread can compute the matching cost for one pixel and one disparity candidate, or one thread can compute the best disparity for one pixel over a small search range. We will compare different layouts to see which gives better memory access patterns.

The CPU version will be implemented first as a correctness baseline. The CUDA version will then be optimized by considering thread block size, coalesced memory accesses, and possibly shared memory tiling for local image patches. If the kernel is memory-bound, shared memory may help reduce repeated global memory reads. If it is compute-bound, we will focus more on occupancy and reducing unnecessary work.

5 MPI Implementation Plan

For MPI, we plan to distribute the workload across camera streams or image partitions. In the simplest version, each MPI process will handle one camera stream and run the local preprocessing and CUDA kernel on its assigned frames. After local processing, each process will send intermediate data to a root or fusion process. This intermediate data may be a depth map, feature list, or tracked object state.

We will start with a simple blocking communication pattern to verify correctness. After that, we may test non-blocking MPI communication to see whether communication can overlap with GPU computation. The key question we want to answer is whether adding more processes improves throughput or whether communication and synchronization reduce the benefit.

6 Profiling and Performance Analysis Plan

We will evaluate performance using several metrics. The main metrics will be total runtime per frame, frames per second, CUDA kernel runtime, CPU baseline runtime, and MPI scaling efficiency. We will also measure how performance changes with image size, number of cameras, number of MPI processes, and size of the stereo/disparity search space.

For CUDA profiling, we will measure kernel time and memory throughput. We will compare the CPU and GPU implementations to report speedup. For MPI, we will run strong scaling experiments where the total workload stays fixed while the number of processes increases. If time allows, we may also run weak scaling experiments where each process gets its own camera stream or image chunk. These experiments should help us understand whether our system is limited by computation, memory bandwidth, or communication overhead.

7 Timeline

Week 1: Build the CPU baseline pipeline. Load image sequences, implement preprocessing, and choose the first main computational kernel.

Week 2: Implement the first CUDA version of the kernel. Verify correctness against the CPU baseline and measure initial speedup.

Week 3: Optimize the CUDA kernel. Experiment with block sizes, memory layout, and shared memory if useful. Start collecting profiling data.

Week 4: Add MPI support. Distribute camera streams or image chunks across processes and implement the first fusion/collection step.

Week 5: Run scaling experiments. Measure runtime, FPS, speedup, and communication overhead for different numbers of processes.

Week 6: Finalize results, clean up code, produce plots, and write the final report.

References

Mouna Afif, Yahia Said, and Mohamed Atri. Computer vision algorithms acceleration using graphic processors nvidia cuda. *Cluster Computing*, 2020.

Gwangbin Bae, Ignas Budvytis, and Roberto Cipolla. Multi-view depth estimation by fusing single-view depth probability with multi-view geometry. In *CVPR*, 2022.

Enric Cervera. Gpu-accelerated vision for robots: Improving system throughput using opencv and cuda. *IEEE Robotics & Automation Magazine*, 2020.

Jesse Patrick Harvey. Gpu acceleration of object classification algorithms using nvidia cuda. Master's thesis, Rochester Institute of Technology, 2009.

Jing Huang, Sean P. Ponce, Seung In Park, Yong Cao, and Francis Quek. Gpu-accelerated computation for robust motion tracking using the cuda framework. 2007.

Yao Yao, Zixin Luo, Shiwei Li, Tian Fang, and Long Quan. Mvsnet: Depth inference for unstructured multi-view stereo. In *European Conference on Computer Vision*, 2018.